Understanding Kubernetes Architecture

Kubernetes Production · Lesson 1

Built from the official Kubernetes documentation

`kubernetes.io/docs`

Learning Objectives

By the end of this lesson you will be able to:

- Describe what Kubernetes is and the problems it solves
- Identify the **control plane** components and their roles
- Identify the **node (worker)** components and their roles
- Explain how a Pod request flows through the cluster
- Distinguish vanilla Kubernetes, distributions and managed offerings

> Reference: *Kubernetes Components* — kubernetes.io/docs/concepts/overview/components

1.1 What Is Kubernetes?

What Is Kubernetes?

Kubernetes (K8s) is an open-source platform for **automating deployment, scaling, and management** of containerized applications.

It provides:

- **Service discovery & load balancing** — stable names and IPs for Pods
- **Storage orchestration** — mount local or cloud storage automatically
- **Automated rollouts & rollbacks** — declarative desired state
- **Self-healing** — restarts, reschedules, and replaces failed containers
- **Secret & configuration management** — without rebuilding images

> Kubernetes is *declarative*: you describe the **desired state**, controllers continuously reconcile the **actual state** to match it.

The Cluster: Two Planes

A Kubernetes **cluster** is a set of machines (**nodes**) organized into:

Plane	Runs	Hosts user Pods?
Control plane	Cluster "brain": API, scheduling, controllers, etcd	No (by default)
Worker plane	User workloads via kubelet + runtime	**Yes**

- A production cluster runs **multiple control-plane nodes** for high availability.
- The control plane makes **global decisions**; nodes **run the work**.

1.2 Control Plane Components

Control Plane Components

Component	Role
kube-apiserver	Exposes the Kubernetes HTTP API; front door to the cluster
etcd	Consistent, highly-available key-value store for **all** cluster data
kube-scheduler	Assigns unscheduled Pods to suitable nodes
kube-controller-manager	Runs controllers that drive actual → desired state
cloud-controller-manager	Integrates with a cloud provider *(optional)*

> Everything talks to the cluster **through the apiserver** — it is the only component that reads/writes etcd directly.

kube-apiserver & etcd

kube-apiserver

- The RESTful front end for the cluster's shared state
- Validates and processes every request (`kubectl`, controllers, kubelets)
- Horizontally scalable — run several instances behind a load balancer

etcd

- Stores the entire cluster state and configuration
- Strongly consistent (Raft); the **single source of truth**
- **Back it up** — losing etcd means losing the cluster

> CKA tip: you must know how to **snapshot and restore etcd** (Lesson 7).

Scheduler & Controller Manager

kube-scheduler

- Watches for Pods with no assigned node
- Picks a node based on resource requests, affinity, taints, constraints

kube-controller-manager

- A single binary running many controllers, e.g.:
  - **Node controller** — notices and responds to node failures
  - **ReplicaSet / Deployment controllers** — maintain replica counts
  - **Job controller** — runs Pods to completion
- Each controller watches the API and reconciles toward desired state

1.3 Node Components

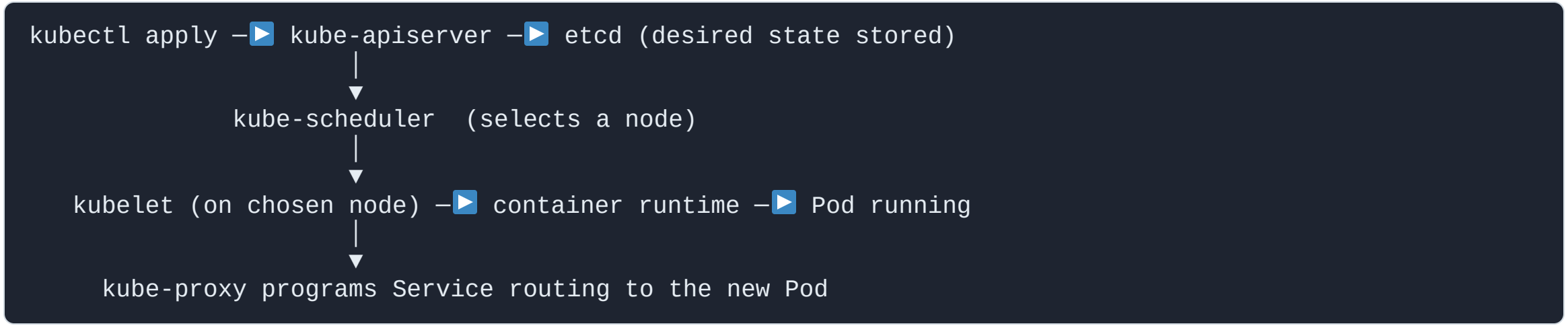
Node (Worker) Components

Component	Role
kubelet	Agent on every node; ensures containers described by PodSpecs are running and healthy
kube-proxy	Maintains network rules so **Services** reach the right Pods *(optional)*
Container runtime	Software that actually runs containers (containerd, CRI-O)

Every node also runs a **CNI plugin** (Calico, Cilium, Flannel…) to provide the Pod network.

> The kubelet only manages containers created by Kubernetes — not arbitrary ones on the host.

How a Pod Gets Scheduled



1. You submit a manifest → apiserver persists it in etcd
2. Scheduler binds the Pod to a node
3. That node's kubelet tells the runtime to start the containers
4. kube-proxy + CNI wire up networking

Cluster Add-ons

Add-ons extend cluster functionality using Kubernetes resources (often in the `kube-system` namespace):

- **DNS** (CoreDNS) — cluster-wide name resolution *(essentially mandatory)*
- **Web UI (Dashboard)** — browser-based management
- **Container Resource Monitoring** — metrics collection (e.g. metrics-server)
- **Cluster-level Logging** — central log storage

> Vanilla Kubernetes ships **no** default CNI or Ingress controller — you choose and install them.

1.4 Vanilla, Distributions & Managed

Vanilla, Distributions & Managed K8s

Flavor	What it is	Examples
Vanilla	Upstream Kubernetes installed directly (e.g. `kubeadm`)	kubeadm, Kubespray
Distribution	Vanilla + ecosystem add-ons + vendor support, often a version behind	OpenShift, Rancher, Charmed K8s
Managed	Cloud provider runs the control plane for you	EKS, AKS, GKE
Learning	All-in-one, single host	minikube, kind, k3s

- The **CKA exam** uses vanilla, `kubeadm`-built clusters.
- Distributions trade some flexibility for integration and support.

Key Takeaways

- Kubernetes = **declarative** orchestration of containers; controllers reconcile desired vs actual state
- **Control plane**: apiserver, etcd, scheduler, controller-manager (+ optional cloud-controller-manager)
- **Node**: kubelet, kube-proxy, container runtime, CNI
- **All traffic flows through the apiserver**; only it touches etcd
- Know the difference between **vanilla / distribution / managed**

End of Lesson 1

Next: Lesson 2 — Requirements & Environment

`kubectl get nodes` · `kubectl get componentstatuses` · `kubectl get pods -n kube-system`